

# ***Design and Analysis of Algorithms***

## **Merge Sort & Recurrence Relation**

National University of Computer and Emerging Sciences,  
Islamabad

# Merge sort

- The *merge sort* algorithm closely follows the **divide-and-conquer paradigm**. Intuitively,
- it operates as follows.
- **Divide:** Divide the  $n$ -element sequence to be sorted into two subsequences of  $n/2$  elements each.
- **Conquer:** Sort the two subsequences recursively using merge sort.
- **Combine:** Merge the two sorted subsequences to produce the sorted answer.

## *Merge Sort (Divide and Conquer)*

**MERGE-SORT**  $A[1 \dots n]$

1. If  $n = 1$ , done.
2. Recursively sort  $A[1 \dots \lceil n/2 \rceil]$   
and  $A[\lceil n/2 \rceil + 1 \dots n]$ .
3. “*Merge*” the 2 sorted lists.

*Key subroutine:* **MERGE**

## Key Operation

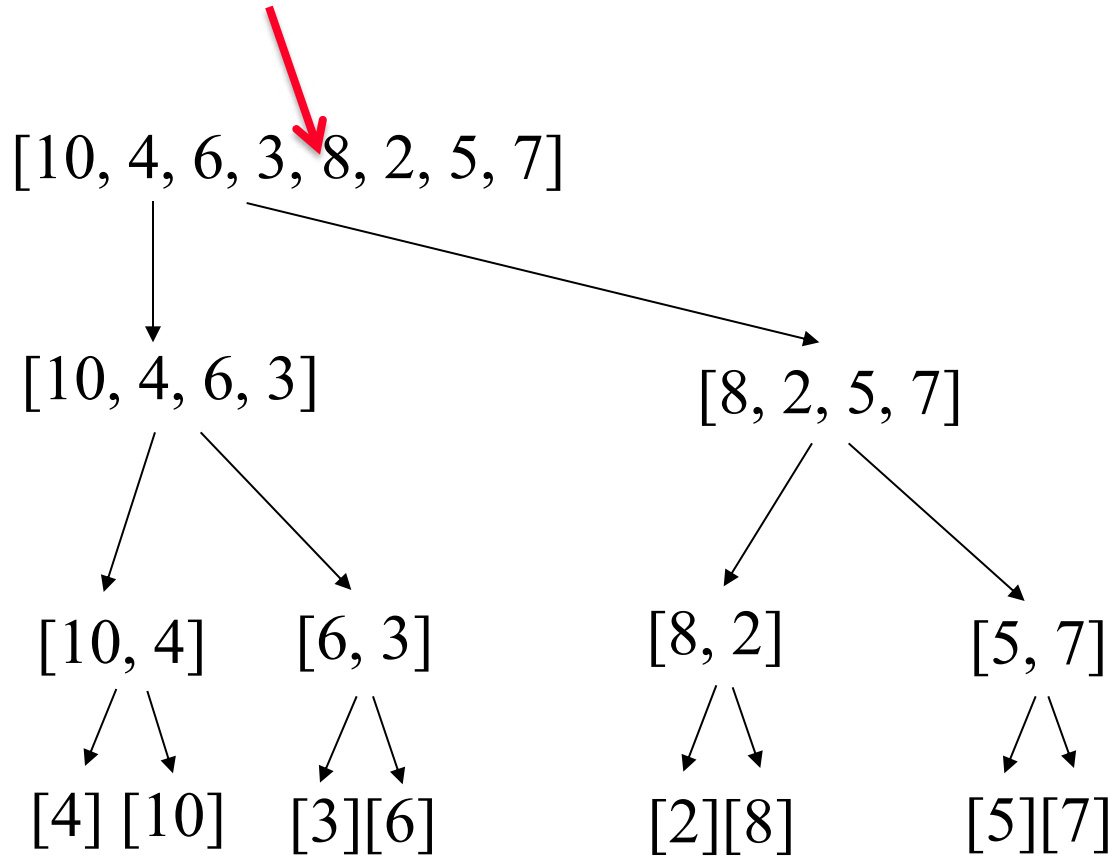
- The key operation of the merge sort algorithm is the merging of two sorted sequences in the “combine” step.
- We merge by calling an auxiliary procedure
- **MERGE** ( $A, p, q, r$ ), where  $A$  is an array and  $p, q$ , and  $r$  are indices into the array such that  $p \leq q < r$ .
- The procedure assumes that the subarrays  $A[p..q]$  and  $A[q+1..r]$  are in sorted order.
- It **merges** them to form a single sorted subarray that replaces the current subarray  $A[p..r]$

# *Visualization MergeSort*

- <https://opendsa-server.cs.vt.edu/embed/mergesortAV>

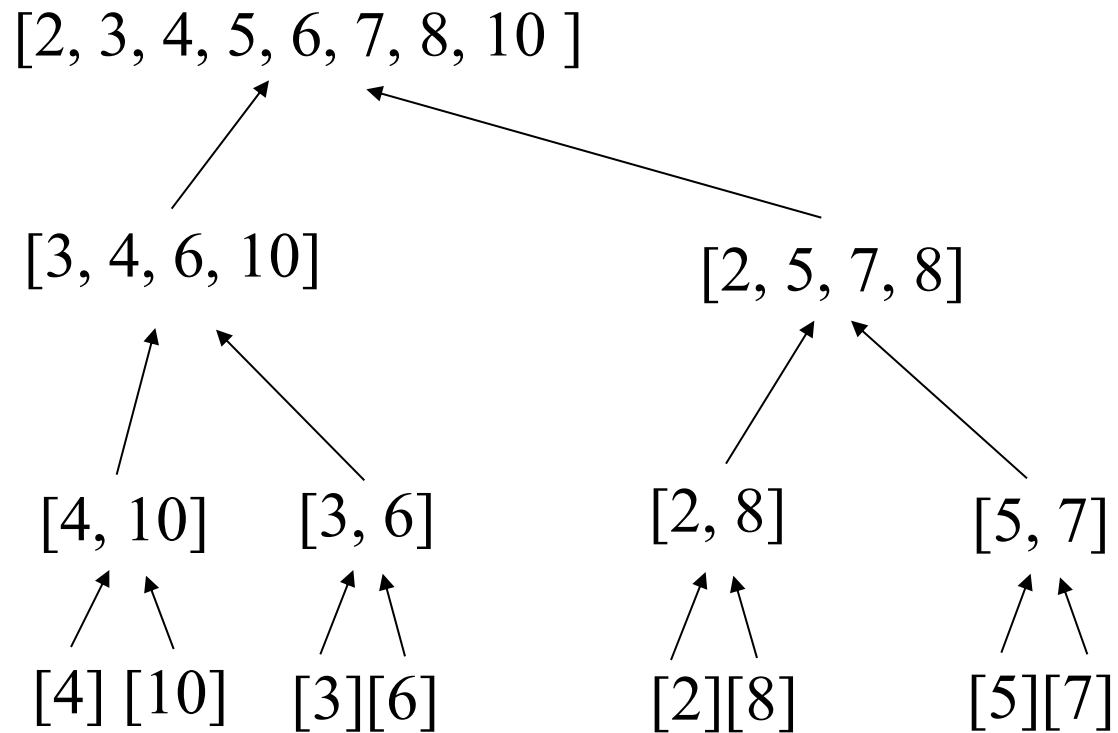
## Example

- Partition into lists of size  $n/2$



## *Example Cont'd*

- Merge**



**MERGE**( $A, p, q, r$ )

```
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```

Reference: Pg#31  
Introduction to algorithms



# Merge-Sort

MERGE-SORT( $A, p, r$ )

```
1  if  $p < r$   
2       $q = \lfloor (p + r) / 2 \rfloor$   
3      MERGE-SORT( $A, p, q$ )  
4      MERGE-SORT( $A, q + 1, r$ )  
5      MERGE( $A, p, q, r$ )
```

# ***Analysis of Merge Sort***

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c , \\ aT(n/b) + D(n) + C(n) & \text{otherwise .} \end{cases}$$

**Divide:** The divide step just computes the middle of the subarray, which takes constant time. Thus,  $D(n) = \Theta(1)$ .

**Conquer:** We recursively solve two subproblems, each of size  $n/2$ , which contributes  $2T(n/2)$  to the running time.

**Combine:** We have already noted that the MERGE procedure on an  $n$ -element subarray takes time  $\Theta(n)$ , and so  $C(n) = \Theta(n)$ .

## Worst-case running time $T(n)$ of Merge sort:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T(n/2) + \Theta(n) & \text{if } n > 1. \end{cases}$$

- Merge sort, with its  $\theta(n \lg n)$  running time, outperforms insertion sort, whose running time is  $\theta(n^2)$ , in the worst case.

# ***Analysis of Merge Sort***



# Recurrences

- The expression:

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + cn & n > 1 \end{cases}$$

is a *recurrence*.

- **Recurrence:** an equation that describes a function in terms of its value on smaller functions

# Recurrence Examples

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

$$s(n) = \begin{cases} 0 & n = 0 \\ n + s(n-1) & n > 0 \end{cases}$$

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + c & n > 1 \end{cases}$$

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

# Example

**Function(int number)**

if number<=1

then return;

else

Function(number/2)

Function(number/2)

for(i=1 to number )

Display i;

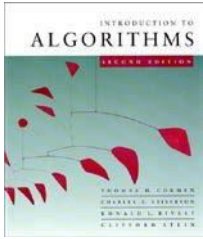
***It follows that***

$$T(n) = 2 T(n/2) + \textcircled{cn}$$

number of sub-problems

Sub-problem size

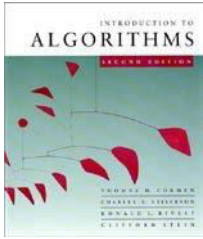
work done dividing and combining



# Recursion tree

Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.

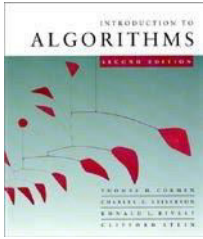




## *Recursion tree*

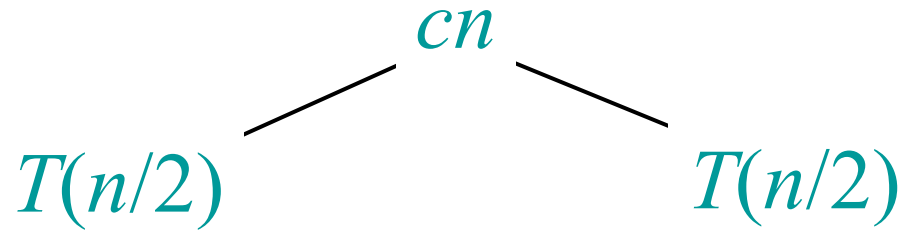
Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.

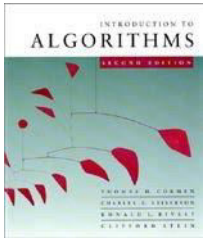
$$T(n)$$



## *Recursion tree*

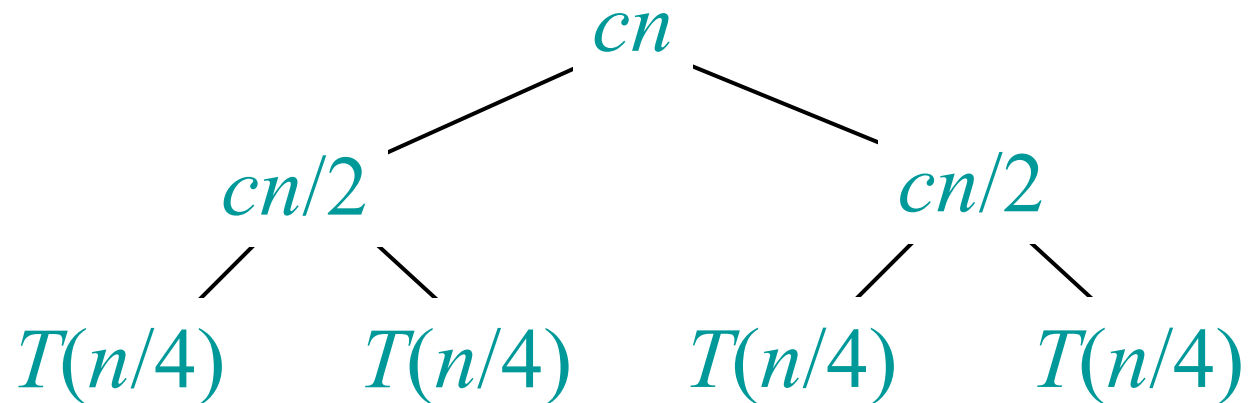
Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.

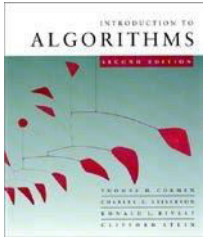




## Recursion tree

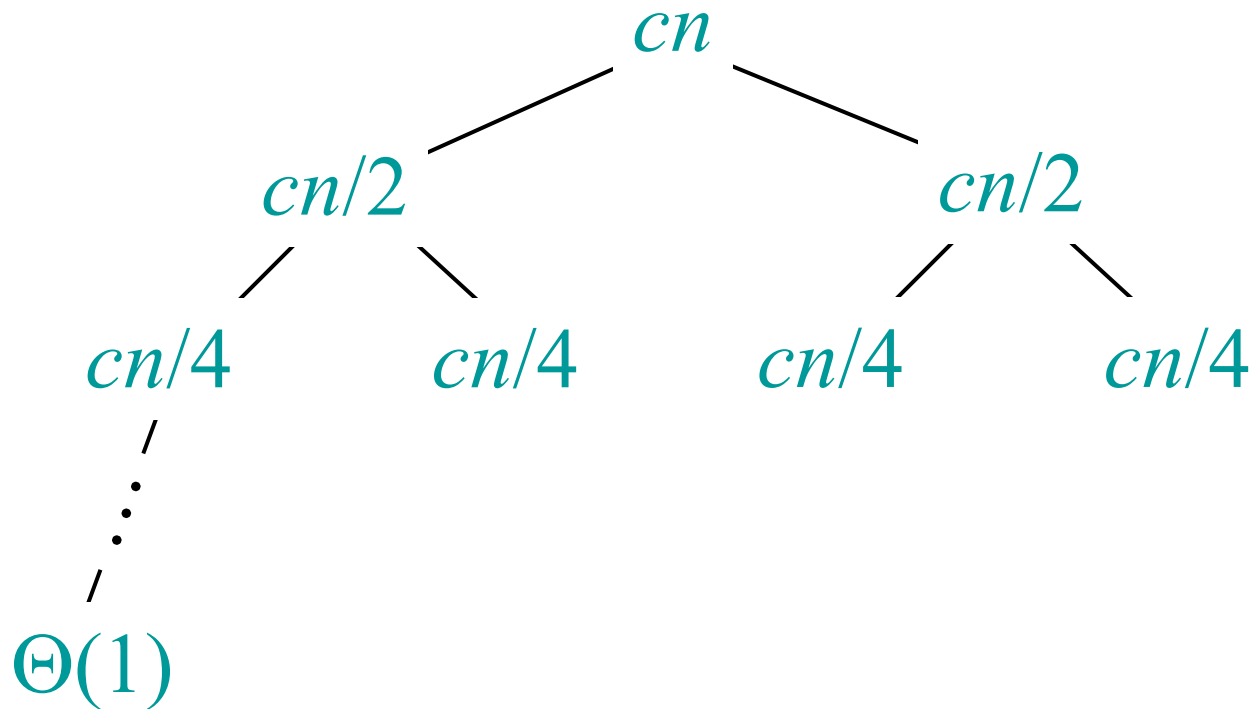
Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.

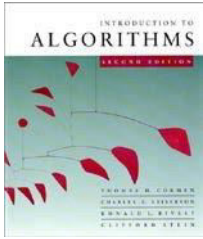




## Recursion tree

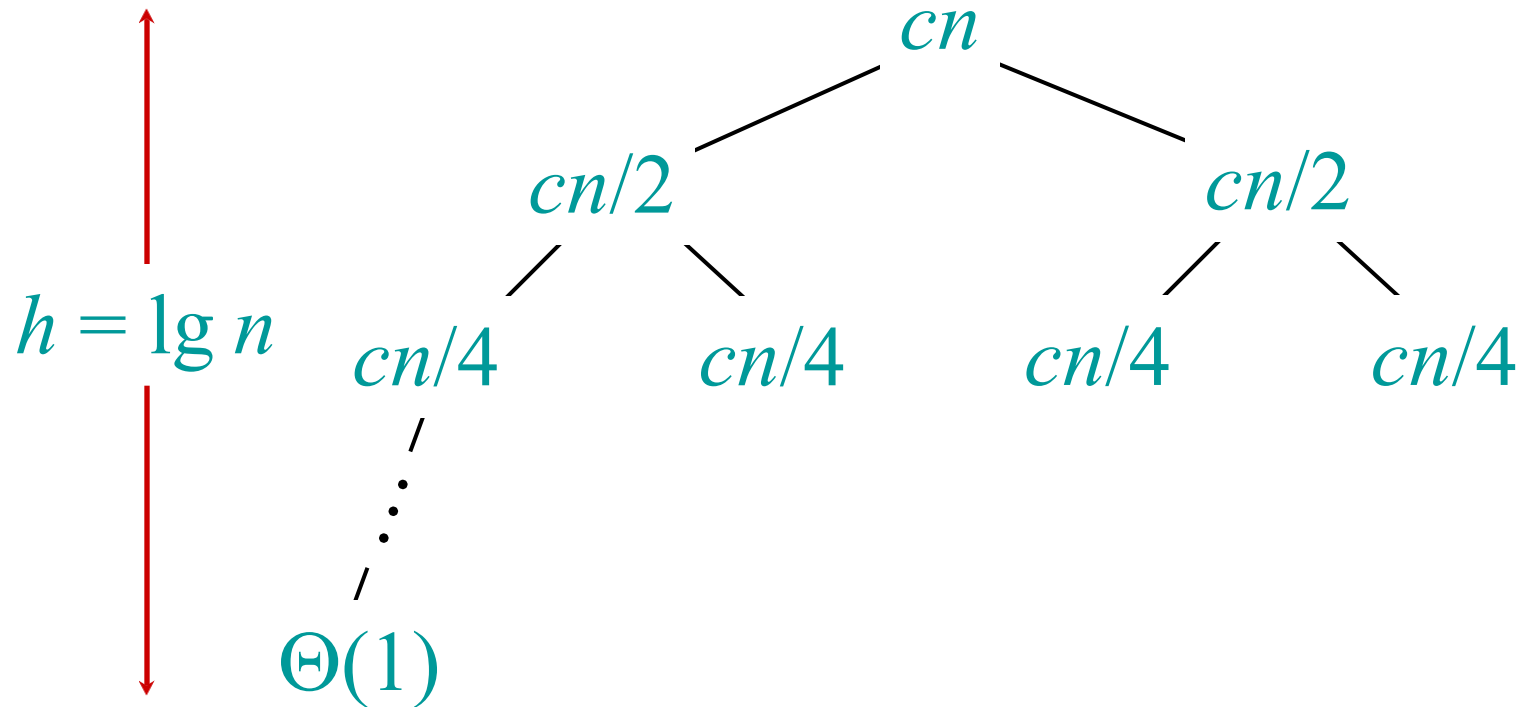
Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.

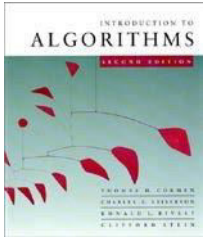




## Recursion tree

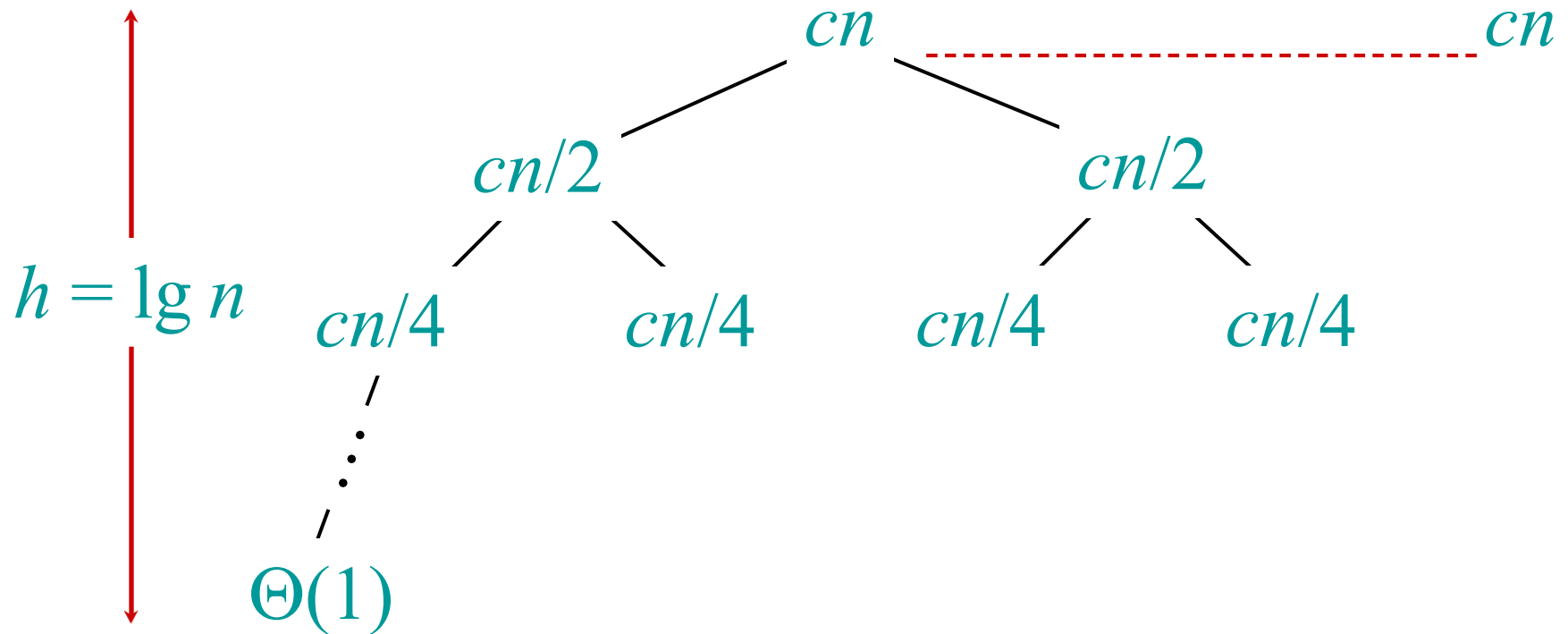
Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.

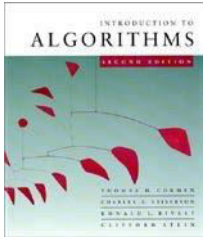




## Recursion tree

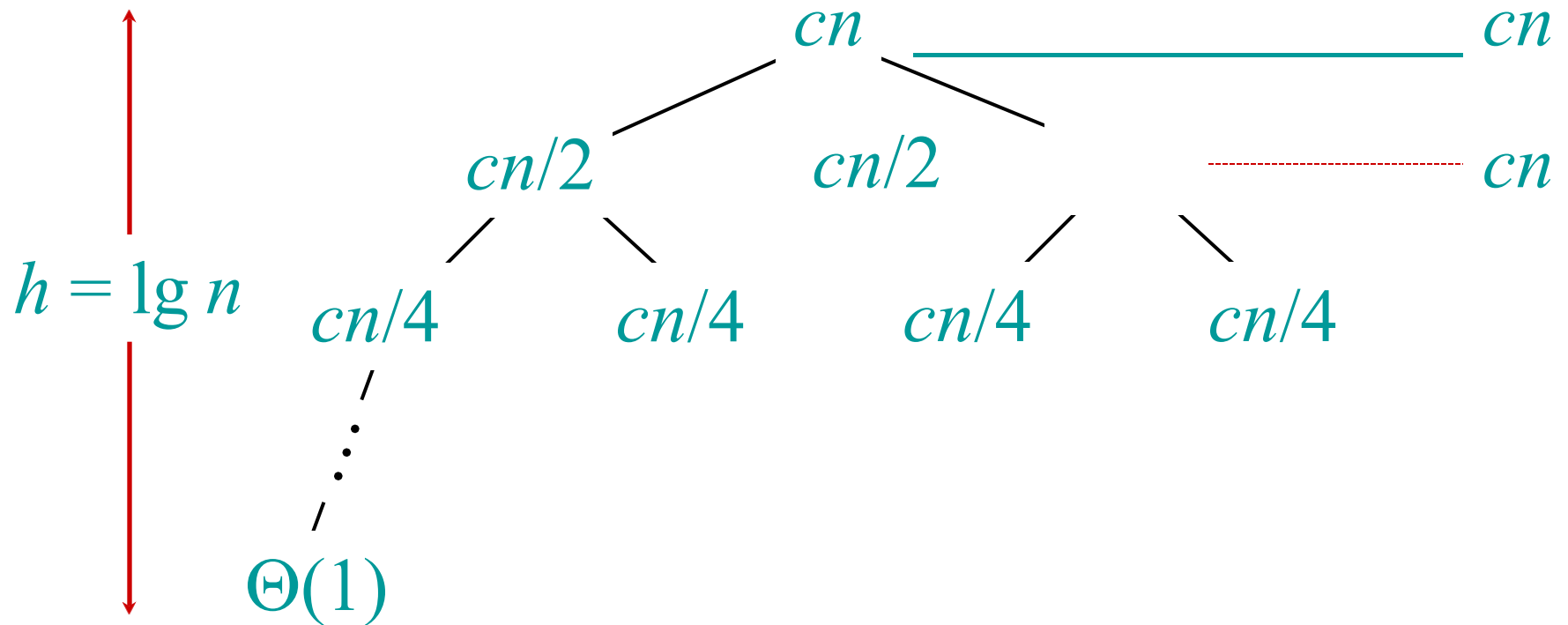
Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.

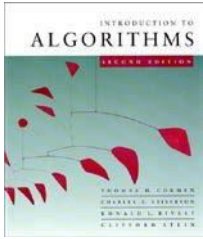




## Recursion tree

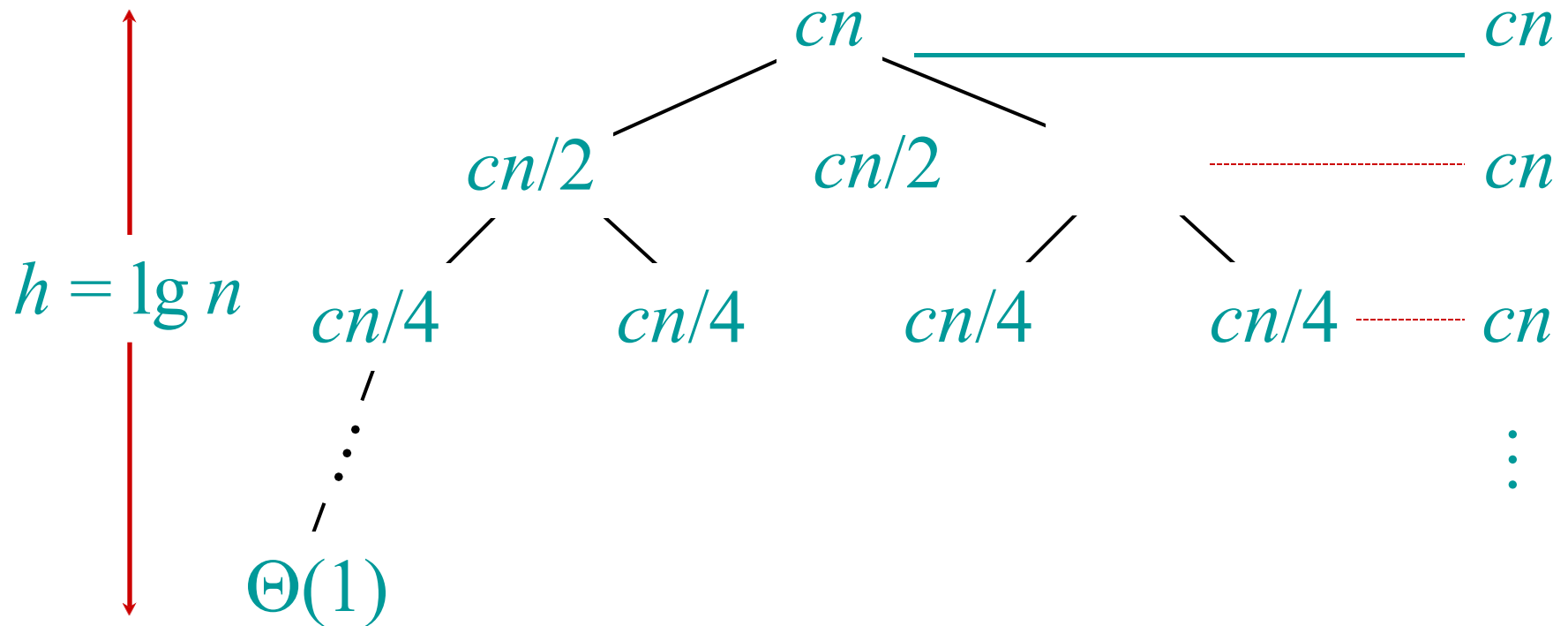
Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.



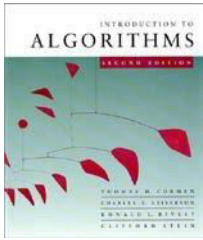


## Recursion tree

Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.



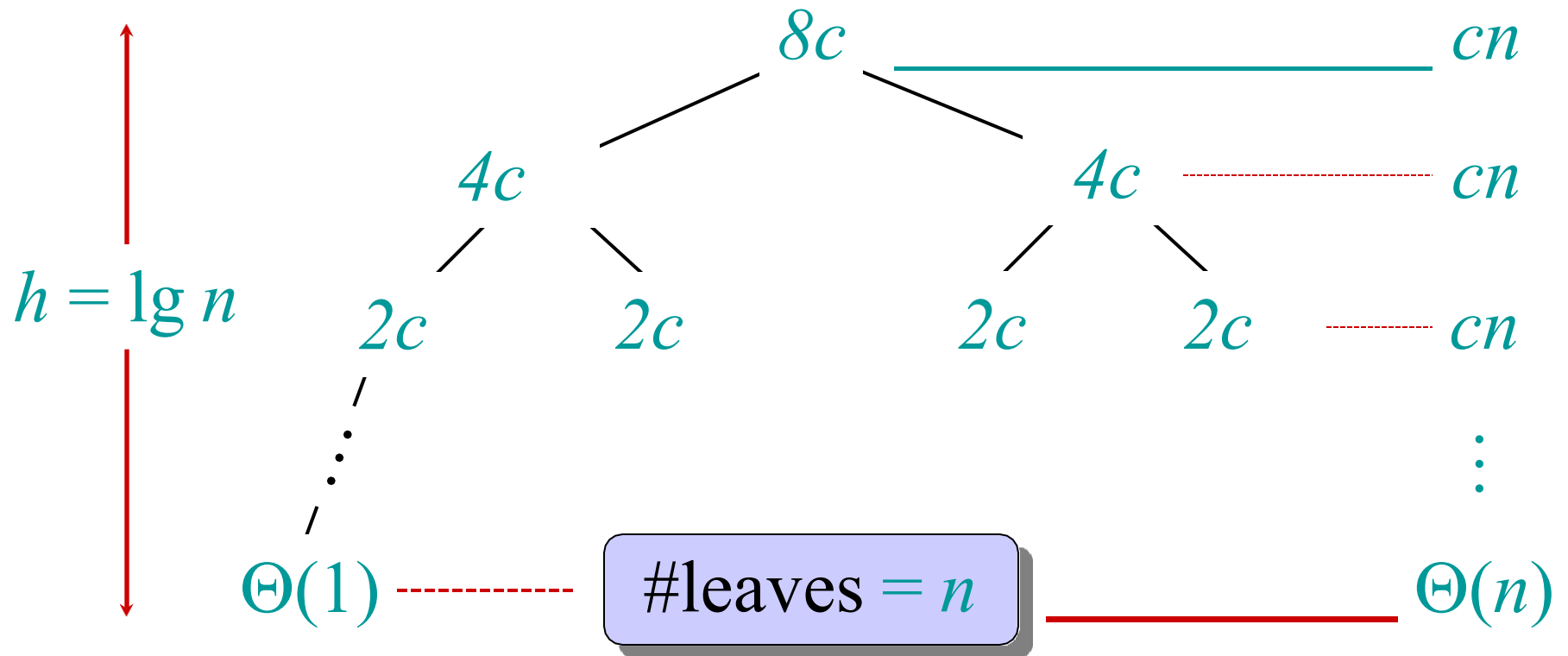


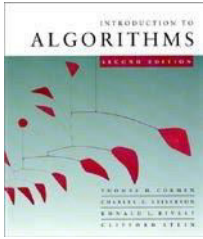


$$n=8$$

## Recursion tree

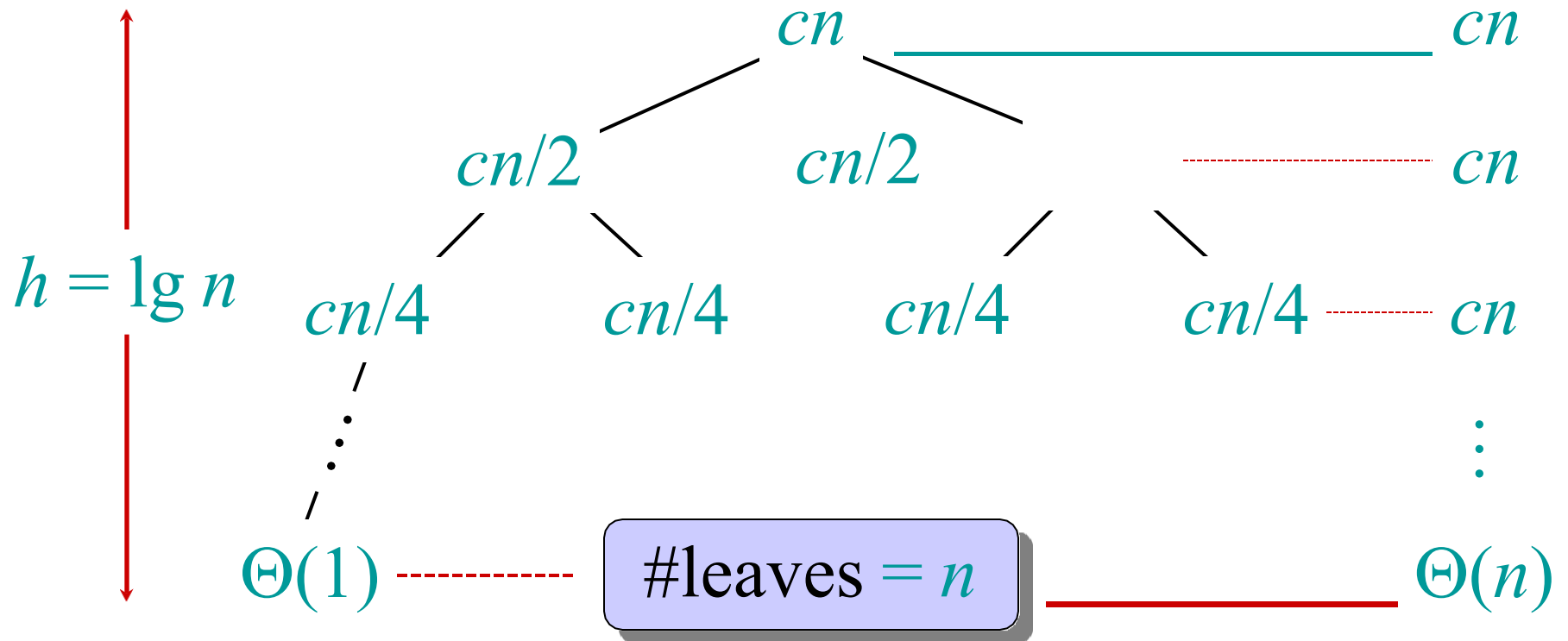
Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.

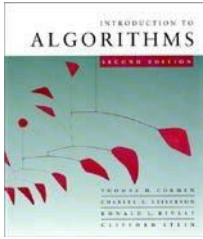




## Recursion tree

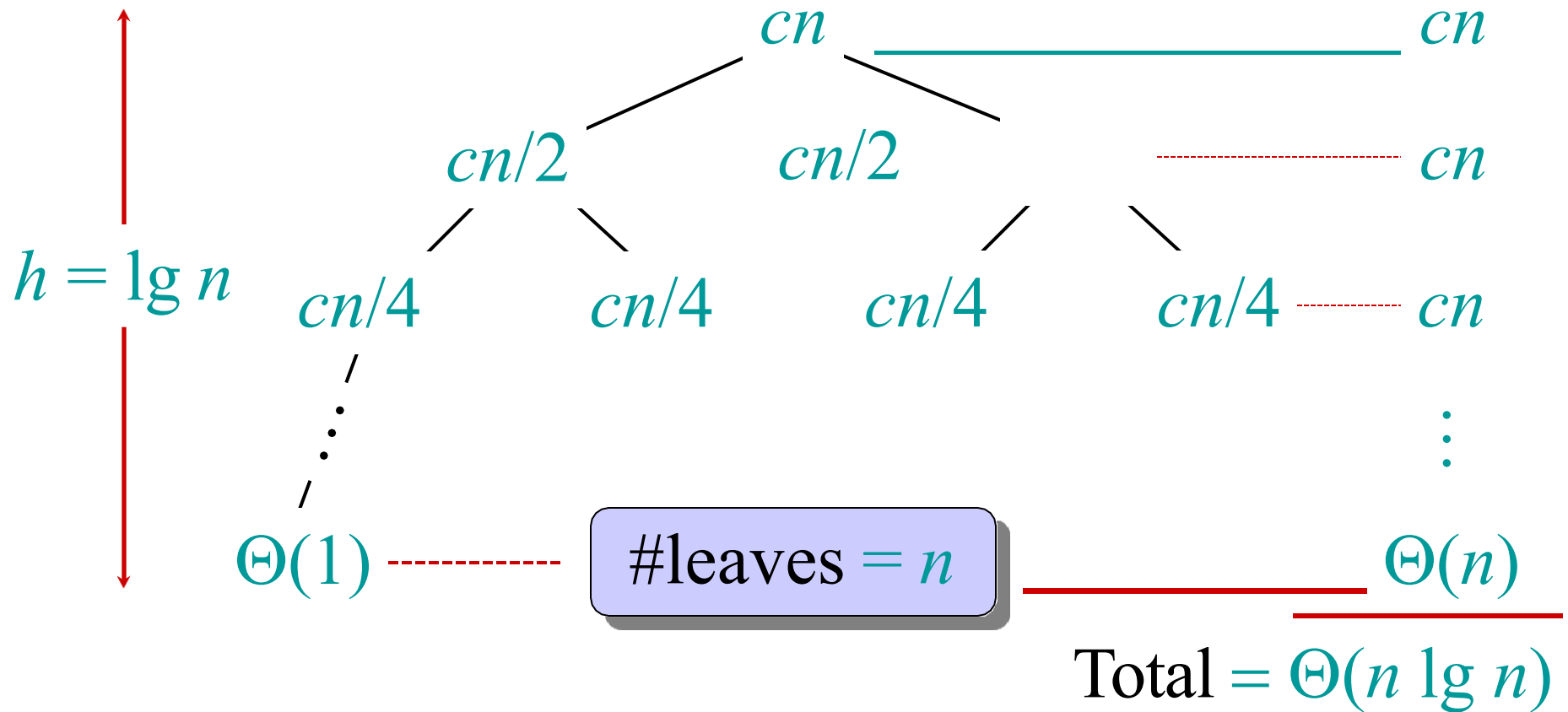
Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.



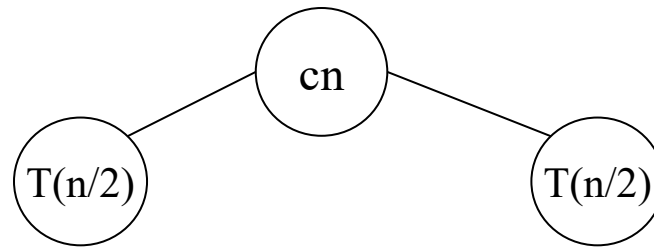


## Recursion tree

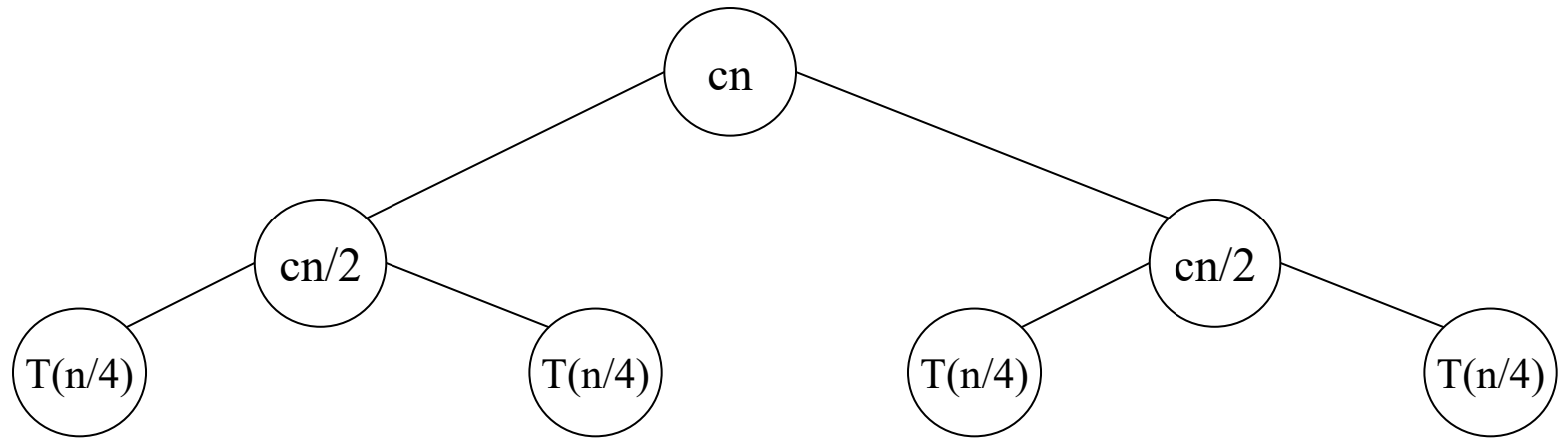
Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.



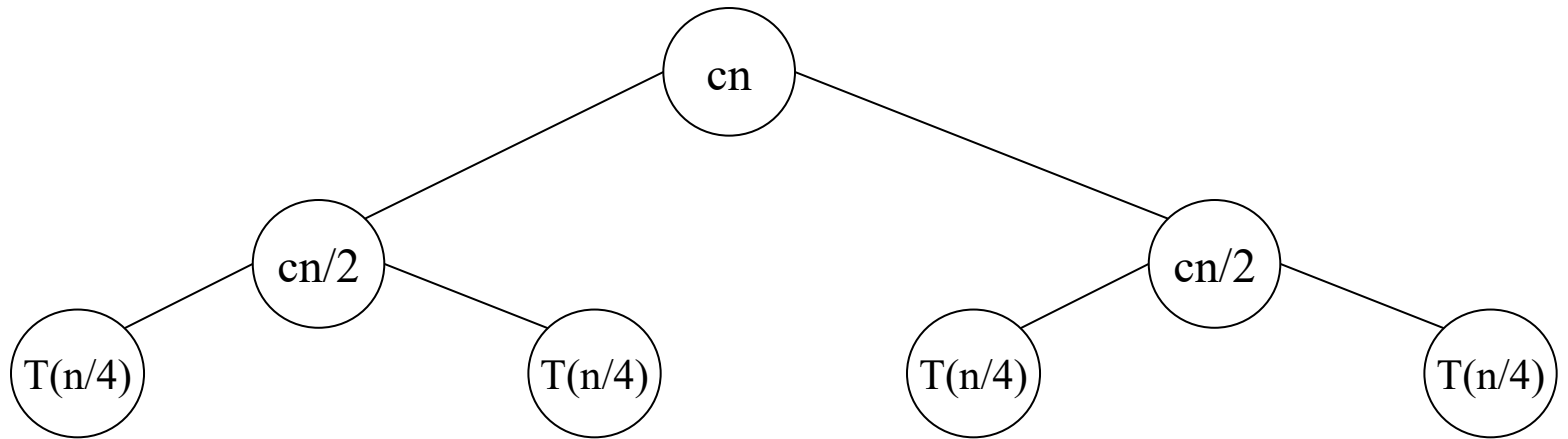
# Recursion Tree for Algorithm



# Recursion Tree for Algorithm

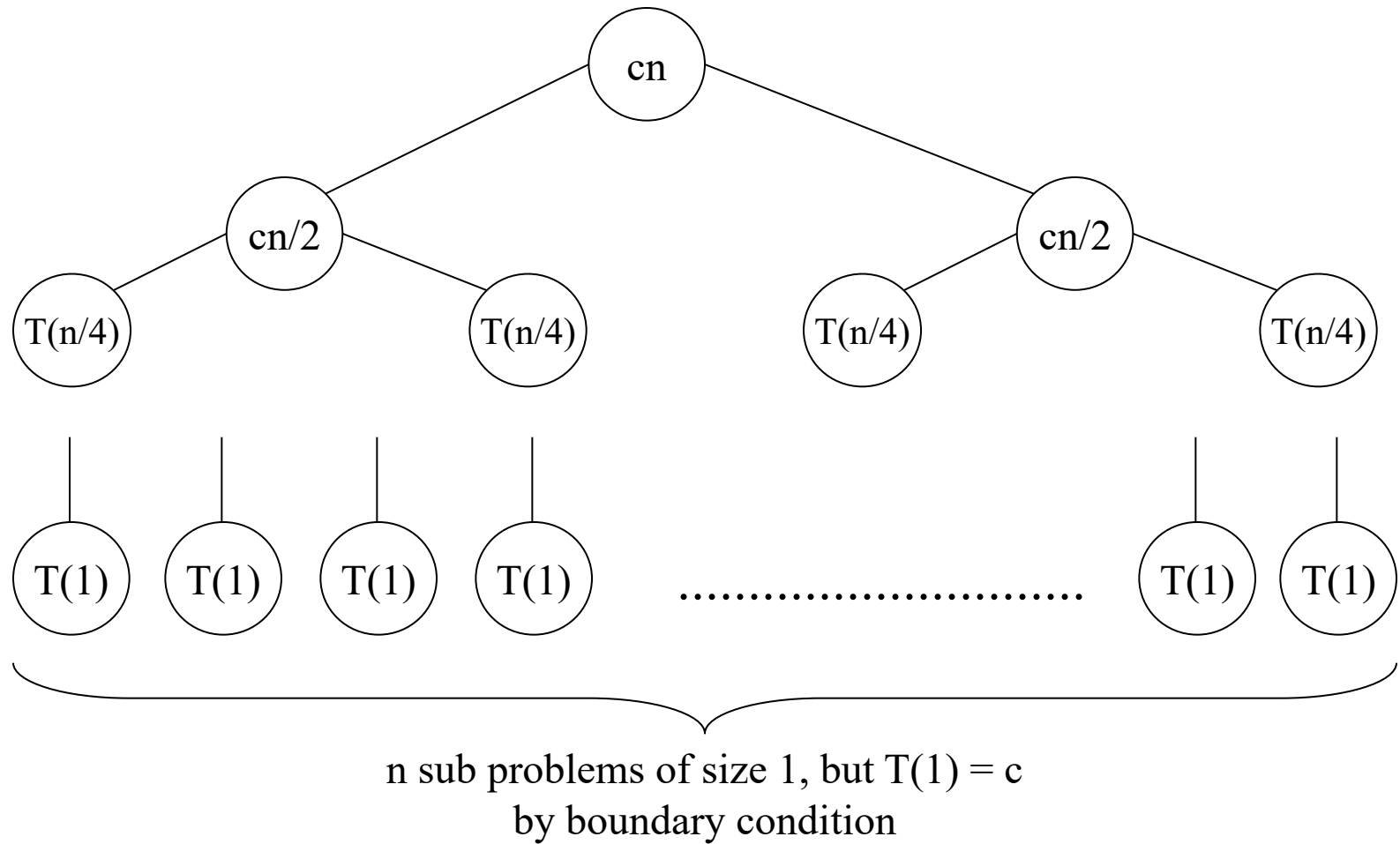


## Recursion Tree for Algorithm

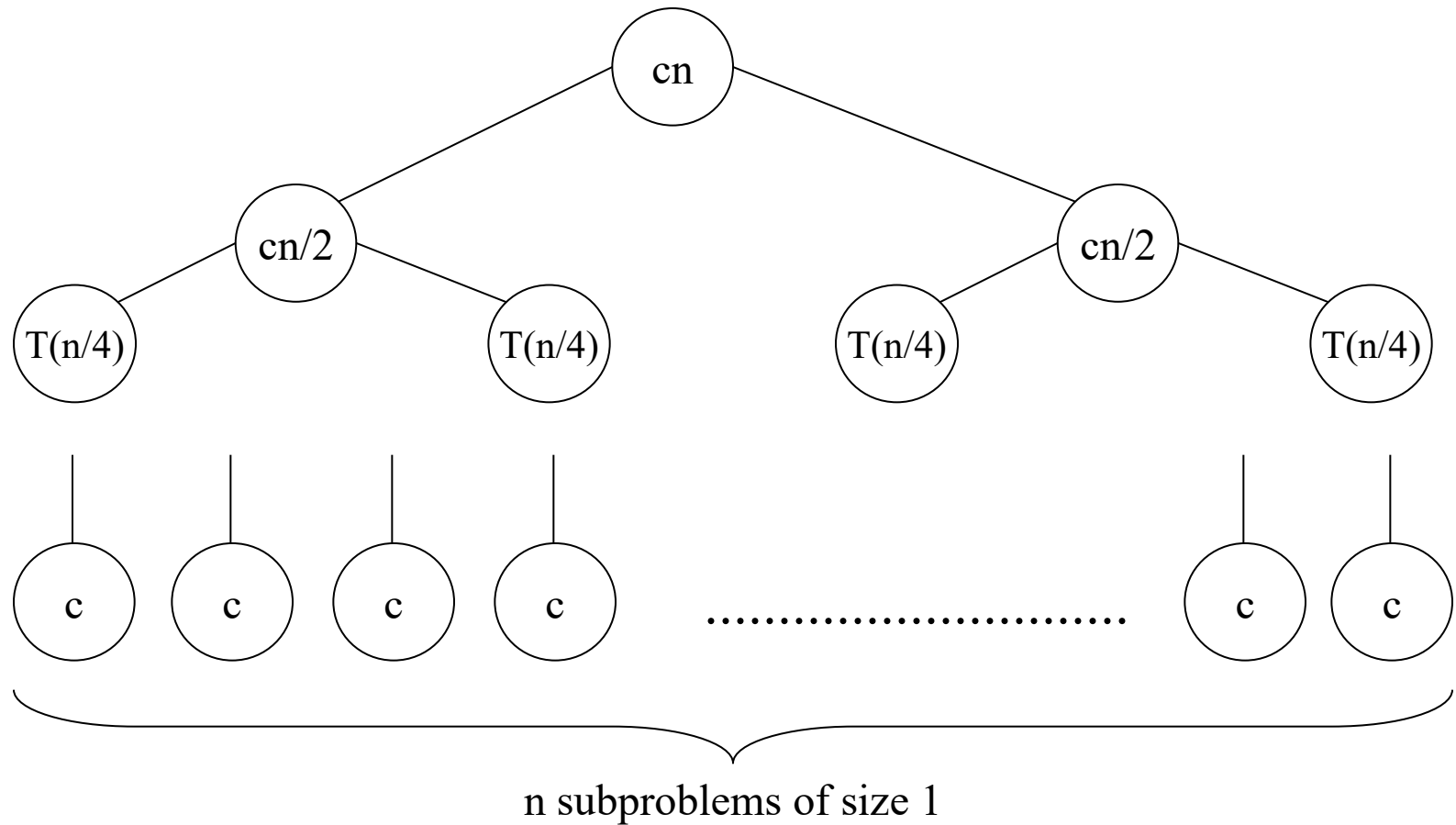


Eventually, the input size (the argument of  $T$ ) goes to 1, so...

# Recursion Tree for Algorithm



# Recursion Tree for Algorithm





# Recursion Tree for Algorithm

| level | nodes/<br>level | cost/<br>level |
|-------|-----------------|----------------|
| 0     | $2^0 = 1$       | cn             |
| 1     | $2^1 = 2$       | cn             |
| 2     | $2^2 = 4$       | cn             |

.

.

.

N-1                       $2^{N-1} = n$

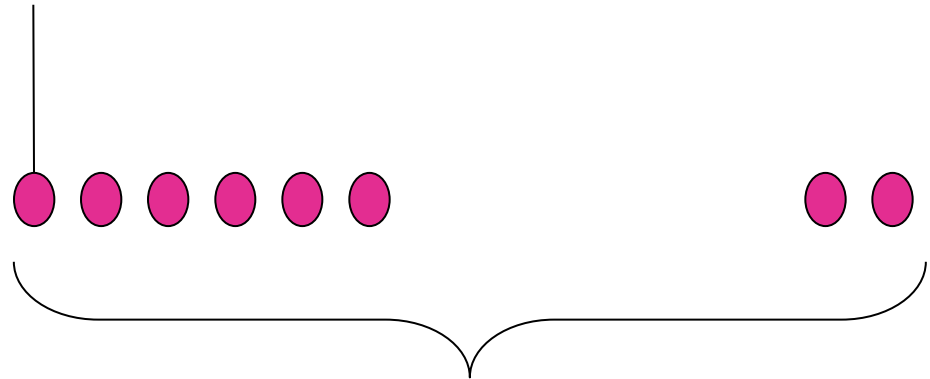
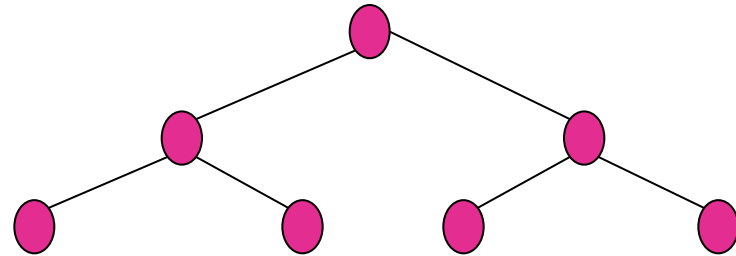
Since  $2^{N-1} = n$ ,

$N-1 = \lg(n)$

levels =  $N = 1 + \lg(n)$

$T(n) = \text{total cost} = (\text{levels})(\text{cost/level})$

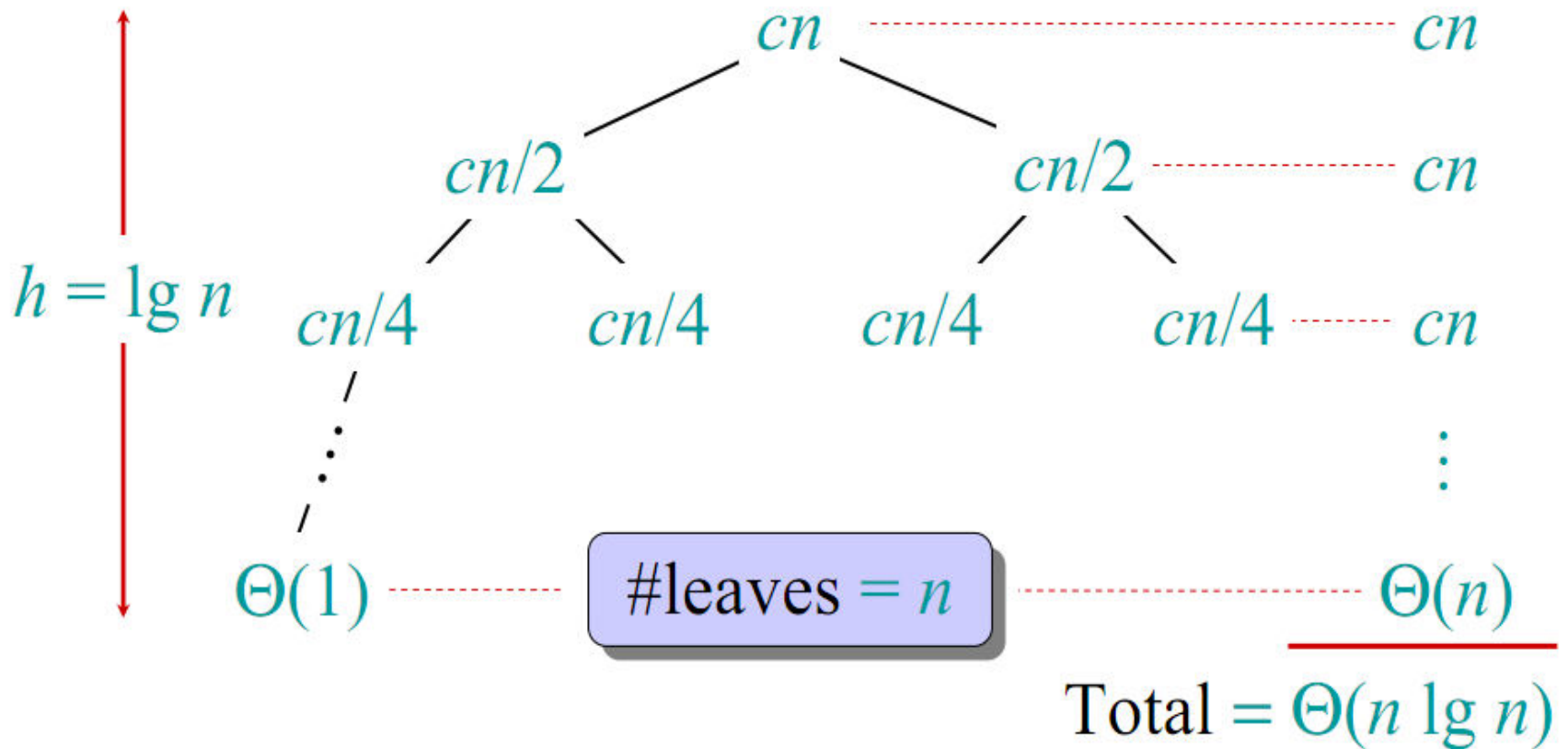
$T(n) = cn [1 + \lg(n)] = O(n \lg(n))$



n nodes at level N-1

# Visual Representation of the Recurrence for Merge Sort

Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.



## ***Time Complexity (Using Master Theorem)***

- **Recurrence Relation**

$$T(n) = 2T(n/2) + n$$

**Using Master Theorem applying case 2:**

$$\Theta\left(n^{\log_b a} \log n\right)$$

**So time complexity is  $O(n \log n)$**

- $\Theta(n \lg n)$  grows more slowly than  $\Theta(n^2)$
- Therefore, **merge sort asymptotically beats insertion sort in the worst case.**
- In practice, merge sort beats insertion sort for  $n \geq 3$

# ***Sorting algorithms***

- **Selection and bubble sort have quadratic best/average/worst-case performance**
- **Insertion sort has quadratic average-case and worst-case performance**
- **The faster comparison based algorithm ?  
 $O(n \log n)$**
- **Mergesort and Quicksort**

# *Solving Recurrences*

- **The substitution method**
  - A.k.a. “making a good guess method”
  - Guess the form of the answer, then use mathematical induction to find the constants and show that the solution works
  - **Example:**
    - $T(n) = 2T(n/2) + n \quad \square \quad T(n) = O(n \lg n)$